Addis Ababa University — School of Information Technology and Engineering
Software Testing and Validation — Final Project

Test Plan

Clinic Booking System

Group members	Biruk Mulatu Kibret — ATE/9686/14 Yesehak Abraham Mesfin — ATE/8291/14 Hawa Nursefa Fujaga — ATE/7005/14 Habtamu Zeleke Muluneh — ATE/7735/14 Fenet Teshome Argeta — ATE/9860/14
Instructor	Abel Tadesse
Date	4 September 2026 (revised 6 September 2026)

1. Scope

This plan covers the testing effort for the Clinic Booking System, a small Spring Boot web application built as the vehicle for this course's testing techniques. Per the assignment brief, the application itself earns no marks; this document plans the testing of its business logic and user interface.

The four course rules below (1, 2, 3, 3b) are the primary vehicle for Part B's formal test design — each is deliberately shaped to feed exactly one technique. During development the application was extended with further business rules (F–L) and subsystems; these are tested to the same standard, with the same techniques, and are included in the coverage and metric figures. They are listed here for completeness; the detailed technique working in the test design document stays focused on rules 1–3b so the derivation is easy to follow.

1.1 In scope

Core course rules (Part B vehicle):

- **Rule 1 — consultation fee by age** (`core.FeeCalculator`): three age bands (CHILD, ADULT, SENIOR) plus rejection of out-of-range ages. → equivalence partitioning, boundary value analysis.
- **Rule 2 — booking eligibility** (`core.BookingPolicy`): a three-condition decision table (slot free, no outstanding balance, ≥2 hours notice) evaluated in strict priority order. → decision table.
- **Rule 3 — appointment lifecycle** (`core.AppointmentStateMachine`): the appointment state machine (now 7 states × 7 events, 10 valid and 39 invalid transitions after the extensions). → state transition testing.
- **Rule 3b — late cancellation fee**: a 24-hour guard condition on top of the CANCEL transition. → boundary value analysis.

Extension rules (tested with the same four techniques; see the test design document §5 and the named test classes):

- **Rule F** — patient-account registration (`PatientRegistrationPolicy`): five-condition decision table + BVA on password length and age.
- **Rule G** — doctor daily-capacity load band (`DoctorLoad`): EP over AVAILABLE / NEARLY_FULL / FULL + BVA around the limit.
- **Rule H** — 24-hour appointment reminder (`ReminderPolicy`): decision table + BVA on the 24-hour window.
- **Rule I** — insurance coverage (`CoverageCalculator`): EP + BVA on the 0–100 % range.
- **Rule J** — no-show suspension, three strikes in 90 days (`SuspensionPolicy`): two independent BVA targets (the count and the window edge).
- **Rule K** — reschedule (`ReschedulePolicy`): decision table + BVA on the 2-hour notice + a new state-machine self-loop.
- **Rule L** — waitlist offer expiry (`WaitlistOfferPolicy`): BVA on the 2-hour acceptance window + a new terminal state.

Also in scope:

- The orchestration layer (`AppointmentService`) that wires the rules to repositories, a `Clock` , and a notification service.
- The web layer (login, slot list, booking form, confirmation, my-appointments) and the end-to-end user journeys it supports.
- **Pairwise (all-pairs) testing** of self-service booking over its six independent factors — an additional combinatorial technique (`BookingPairwiseIT` , `docs/pairwise-booking.md`).
- **Mutation testing** (PIT) of `et.aau.clinic.core` — a second quality gate that checks the tests *detect* faults, not just execute lines.
- **Concurrent booking of the same slot** — after DEF-005 this is in scope and covered by `BookingConcurrencyIT` (8 threads racing one slot).
- Continuous integration (GitHub Actions and Jenkins) and the regression it is required to catch.

1.2 Out of scope

- Non-functional testing: load, performance, accessibility, security penetration testing.
- Real SMS delivery — `NotificationService` has only a logging implementation, by design; no real gateway exists to test against.
- The React front end (`frontend/`) — a decorative alternative UI over the same JSON API; it has no automated tests of its own and is not part of the graded suite. The server-rendered Thymeleaf pages are what Selenium drives and JaCoCo measures.
- The JSON API controllers and DTOs under `web/api/` — thin pass-throughs to the already-tested `AppointmentService` ; exercised indirectly by the MockMvc integration tests but with no dedicated unit tests.
- Formal user acceptance testing (UAT) with real clinic staff — there is no such audience for a course project; the Selenium system tests stand in for the acceptance level (see 2.1).

2. Approach

2.1 Levels and techniques

The suite follows the test pyramid: many fast unit tests, fewer integration tests, a small number of system tests. Each level uses the technique(s) it is best suited to:

Level	What it exercises	Formal techniques used
Unit	`core/` pure functions, in isolation; `AppointmentService` orchestration with all collaborators replaced by test doubles	Equivalence partitioning, boundary value analysis, decision table, state transition testing (Part B of this project). Mutation testing (PIT) runs on top of this level.
Integration	`AppointmentService` + real Spring Data JPA repositories + real H2 database; controllers via MockMvc (real HTTP request/response cycle, no browser); one true multi-threaded concurrency test	Confirms the wiring and the repository queries behind the rules above actually work, not just the boolean logic in isolation. Pairwise (all-pairs) testing of the booking outcome sits at this level.
System	Booking journeys through a real headless Chrome browser via Selenium, using the Page Object pattern	End-to-end path testing of the required user journey (log in → view slots → book → see fee on confirmation → cancel), plus journeys for insurance coverage, no-show suspension and reschedule

Acceptance-level testing is represented by the same Selenium journey: for a course project with no external stakeholder to run a separate UAT phase, the system test acting out the specified user journey (per the assignment's own example: "log in then place an order") is treated as the acceptance check that the application meets its use case.

2.2 Test doubles

The design deliberately builds in seams for all four test-double kinds the course requires:

Kind	Where	What it replaces
Stub	A `Clock.fixed(...)` injected into `AppointmentService` in unit tests	The system clock, so the 2-hour and 24-hour boundaries can be hit exactly instead of racing real time
Mock	Mockito `@Mock NotificationService` in `AppointmentServiceTest`	The SMS-sending collaborator, so a confirmation can be `verify()` -ed without sending one
Fake	The H2 in-memory database used by every `*IT` test	A production database — a real, working, but lightweight implementation
Spy	`@SpyBean NotificationService` in `AppointmentServiceIT`	Wraps the real `LoggingNotificationService` so calls can still be verified while the real logging behaviour still runs

2.3 Coverage and quality targets

- **Branch coverage:** at least 80% of `et.aau.clinic.core` , enforced by a JaCoCo `check` rule so `mvn verify` fails the build if it regresses. Actual figure achieved: 100% (134/134 branches) — see the coverage report.
- **Mutation score:** at least 90% on `et.aau.clinic.core` , enforced by a PIT `mutationThreshold` . Actual: 100% (165/165 mutants killed). Mutation testing is not bound to `verify` ; it runs as its own step in both CI pipelines.

3. Entry and exit criteria

3.1 Entry criteria (before a test level is considered ready to run)

- Unit level: the corresponding `core/` class or service method compiles and its test-double seams (`Clock` , `NotificationService`) exist.
- Integration level: unit tests for the same functionality are green; the Spring context starts successfully under the `test` profile against H2.
- System level: the application starts on a random port under `@SpringBootTest` ; the pages under test have the stable element `id` attributes Selenium locators depend on.

3.2 Exit criteria (before the testing effort is considered complete)

- `mvn clean verify` passes: all unit, integration and system tests green, in one run.
- Branch coverage of `et.aau.clinic.core` is at least 80% (JaCoCo gate passing, not just reported).
- Mutation score of `et.aau.clinic.core` is at least 90% (PIT gate passing).
- Every valid and invalid state transition in Rule 3's 7×7 grid has an explicit test.
- Every boundary in the project's domain rules has an explicit test: ages –1, 0, 17, 18, 64, 65, 120, 121; the 2-hour notice boundary; the 24-hour cancellation boundary; the password-length and age edges of Rule F; the doctor-limit edge of Rule G; the 24-hour reminder window (Rule H); the 0/100 % coverage edges (Rule I); the count-of-3 and 90-day-window edges of Rule J; the 2-hour reschedule-notice edge (Rule K); the 2-hour offer-acceptance window (Rule L).
- GitHub Actions is green on the current `main` , and the Jenkins pipeline has been run at least once to a successful build via the provided Docker setup.
- A regression has been deliberately introduced, observed failing in CI, and fixed, with both runs recorded as evidence.
- All defects found during the effort are logged with a status of Closed, or are explicitly carried as residual risk in the test summary report.

4. Risk-based prioritisation

Priority is set by likely impact (mostly: does it produce a wrong fee, a wrong booking outcome, or a corrupted appointment state?) combined with how many conditions/branches the logic has to get right, since more branching means more ways to get it wrong.

Priority	Area	Why
P1 — highest	Rule 2, booking eligibility decision table (`BookingPolicy`)	Three conditions evaluated in a specific priority order; getting the order wrong silently produces the wrong rejection reason or, worse, an approval that should have been a rejection (e.g. double-booking a slot). Highest branch count of the three rules, and the one most directly tied to correctness of a scheduling system.
P1 — highest	Rule 1, fee boundaries (`FeeCalculator`)	Directly affects money charged to a patient. A one-line boundary slip (as demonstrated in the regression, see `docs/regression-demo.md`) silently over- or under-charges real people — exactly the kind of defect BVA exists to catch, and exactly what happened when it was deliberately introduced.
P1 — highest	Concurrent booking of one slot (`AppointmentService` + slot lock)	A check-then-act race on Rule 2's C1: two requests can both read a slot as free and both be created, permanently double-booking it. Found and fixed as DEF-005; guarded by a dedicated 8-thread concurrency test.
P2 — high	Rule 3, state machine invalid transitions	Data-integrity risk: an appointment that can be re-confirmed after cancellation, or attended twice, corrupts clinic records. 39 of 49 state/event pairs must be rejected, so the chance of missing one is real without a systematic (grid-generated, not hand-picked) test list.
P2 — high	Rule J, no-show suspension	Two independent thresholds (a count of 3, a 90-day window) that gate whether a patient can book at all; an off-by-one on either wrongly locks out or wrongly admits a patient.
P2 — high	Rule 3b, late cancellation fee boundary	A second, independent financial boundary (24 hours) layered on top of a valid CANCEL transition; easy to get an off-by-one on `<` vs `≤` here exactly as happened (deliberately) with Rule 1.
P3 — medium	End-to-end web journey (login → book → confirm → cancel)	Lower branch complexity than the rules above, but the only place a wiring mistake (wrong view name, wrong redirect, wrong model attribute) would actually surface to a user — the integration and system tests exist specifically to catch that class of bug.
P4 — lower	Static/read-only pages, login failure messaging	Few branches, low financial/data-integrity impact if imperfect; still covered, but not where test effort concentrates.

5. Schedule

Real dates (compressed relative to the assignment's suggested three-week plan, since the code phases were completed ahead of

the documentation):

Date	Milestone
2026-08-30 (morning)	Phases 1–6: domain, core rules with unit tests, service layer, controllers, integration tests
2026-08-30 (afternoon)	Phases 7–8: Selenium system tests, GitHub Actions, Jenkinsfile
2026-08-30 (evening)	Deliberate regression demo run and caught in CI; DEF-002 found and fixed
2026-08-30 (late)	Jenkins pipeline verified end-to-end against the Docker setup; DEF-003 found and fixed
2026-09-04	This test plan and the remaining three PDF deliverables (test design, defect log + metrics, test summary + reflection) written up from the evidence above
2026-09-05 / 09-06	Application extended with rules F–L and supporting subsystems, each tested to the same standard; pairwise testing added for the booking outcome; mutation testing (PIT) added and wired into both pipelines; DEF-004 and DEF-005 found and fixed (DEF-005 closes the previously-known concurrent-booking gap). All four deliverables revised to the extended scope.
By 2026-09-13	Submission deadline — repository link and four PDFs on Google Classroom

6. Roles

A group of five. Each member owns one of the roles the assignment distributes across a group; the work was reviewed together and everyone can explain any part of the submission.

Role	Responsibility	Owner
Developer / team lead	Application code: domain, core rules, service layer, web layer; branch integration and build	Biruk Mulatu Kibret
Test designer	Deriving the EP / BVA / decision-table / state-transition and pairwise test cases (Part B)	Yesehak Abraham Mesfin
Test automation engineer	Writing and maintaining the unit, integration and Selenium suites and the Page Objects	Hawa Nursefa Fujaga
CI / DevOps	GitHub Actions workflow, Jenkinsfile and Docker setup, the regression demonstration, mutation-testing (PIT) wiring	Habtamu Zeleke Muluneh
Defect & quality manager	Defect log, metrics, coverage and mutation-score reporting, and this test plan and the test summary report	Fenet Teshome Argeta